



tapps-brain

Persistent Memory for AI Coding Agents

An Architecture & Business Reference

Architectural foundations, retrieval mathematics, and the business case for shared cross-session memory across an autonomous agent fleet.

Issued 2026-05-20 · v3 series · PostgreSQL-only persistence plane (ADR-007)

Prepared by NLT Labs · tapps-brain engineering

Contents

Contents	2
1. Executive Summary	4
2. Why Persistent Memory Matters	6
3. System Architecture Overview	7
4. C4 — System Context	8
5. C4 — Container View	10
6. C4 — MCP Server Components	12
7. C4 — Services Layer	14
8. Hexagonal — Ports and Adapters	16
9. Domain Data Model	17
10. Error Taxonomy	19
11. Flow — Recall	20
12. Flow — Remember / Save	21
13. Flow — Atomic Knowledge Graph Event	22
14. Flow — Hive Propagation	23
15. Mathematical Foundations	24
15.1 Okapi BM25	24
15.2 Reciprocal Rank Fusion (RRF)	24
15.3 Composite scoring	24
15.4 Exponential decay (lazy)	25
15.5 FSRS-style stability and difficulty	25
15.6 Bayesian-style confidence update	25
15.7 Bloom filter false-positive bound	25
15.8 Jaccard / TF-IDF for deterministic consolidation	25
15.9 pgvector HNSW geometry	26
16. Data Stores and Schema	27
17. Security & Threat Model	28

17.1 Per-category mitigations	28
18. Observability & the Quality Loop	29
18.1 EWMA anomaly detector	29
18.2 Telemetry surface	29
19. Concurrency and Scale	30
20. Deployment Topology	31
21. Closing — Business Value Summary	32
A. Appendix — Module Dependency Overview	33

1. Executive Summary

What tapps-brain is, and what it eliminates from the cost structure of an AI engineering org.

tapps-brain is a persistent, deterministic memory system for AI coding agents. It preserves the architectural decisions, code conventions, debug findings, and workflow patterns that an agent (and the team it serves) discovers over a sequence of sessions, and surfaces them — ranked, decayed, and de-duplicated — as cheap retrieval context at the start of every subsequent task.

The system is implemented as a single PostgreSQL-backed service that exposes a small agent-facing surface (Model Context Protocol tools and a REST adapter) and a transport-agnostic Python facade. Every operation in the hot path — full-text search, vector retrieval, ranking, decay, deduplication, consolidation, conflict detection — is deterministic and runs without any external LLM call. Optional LLM judges exist for offline quality scoring; they are never invoked during a recall.

Why this matters to the business

AI agents that forget across sessions repeat the same investigations, re-litigate the same architectural debates, and rediscover the same bug patterns. The cost is paid in model tokens, in engineering hours spent re-explaining the codebase, and in the slow drift of the team's collective knowledge into unsearchable transcripts. tapps-brain converts that ephemeral context into a ranked, queryable asset that survives sessions, agents, and projects — without adding an LLM dependency or new vendor surface.

Operating principles

- **Deterministic by design.** Every retrieval is reproducible. No black-box LLM judges in the request path.
- **PostgreSQL-only.** One durable store. pgvector for semantic recall, tsvector for lexical, row-level project / agent isolation, MVCC concurrency.
- **Two interchangeable interfaces.** A deployed HTTP / MCP service for IDE-side agents, and an in-process Python facade for embedded agent loops. Same engine, identical behaviour.
- **Multi-agent native.** Per-agent private memory, declarative groups, and an opt-in Hive layer for cross-agent shared knowledge with namespace isolation.
- **Self-tending corpus.** Exponential decay, deterministic consolidation, and confidence updates keep the recall pool small, ranked, and trustworthy without operator intervention.

Headline characteristics

Deployment

One Docker container per host (HTTP + MCP on :8080, operator MCP on :8090)

Persistence	PostgreSQL 17 + pgvector HNSW + tsvector (ADR-007)
Scale target	200+ concurrent agents per cluster, row-scoped isolation, MVCC
Retrieval	Lexical (Okapi BM25 over tsvector candidates) + dense (pgvector HNSW)
Hybrid fusion	Reciprocal Rank Fusion with deterministic per-query weights
Lifecycle	Lazy exponential decay, deterministic consolidation, FSRS-style reinforcement
Tiers	Architectural 180 d · pattern 60 d · procedural 30 d · context 14 d half-life
Safety	Prompt-injection detection on ingest; integrity-hashed audit log on every mutation
LLM cost in hot path	Zero (offline judges are opt-in for evaluation only)

2. Why Persistent Memory Matters

The business case — what changes when agents stop forgetting.

The unit economics of an AI engineering organisation are dominated by repeat work. A coding agent without persistent memory enters every session in the same naive state: the codebase looks foreign, the team's conventions are invisible, and the failure modes that were debugged yesterday are unknown today. The team compensates by re-supplying context — either by hand (engineer time) or by ballooning the prompt with retrieved files (model tokens). Both costs are paid *per session*, and both scale linearly with the number of agents.

Persistent memory shifts the cost curve. Once a decision is captured with its rationale, the next session retrieves it in a few hundred tokens instead of re-deriving it from a much larger working set. Once a debug session reveals a non-obvious invariant, every subsequent agent that touches the same module sees it. Once an architectural pattern is named, the team — human and machine — stops re-inventing local variants.

Concrete business outcomes

- **Lower per-task model cost.** A focused 1 500-token recall replaces a multi-thousand-token context dump on every turn. Token spend scales with task novelty, not with codebase size.
- **Higher first-pass correctness.** Agents arrive pre-loaded with the constraints, previous fixes, and architectural rules that govern the task. Fewer wrong turns, fewer review cycles, fewer regressions.
- **Onboarding curve approaches zero.** A new agent (or a new human teammate consuming the same memory through MCP) inherits the project's accumulated knowledge from minute one.
- **Institutional knowledge survives team turnover.** Rationale captured by one agent in one session is durable, ranked, and reachable by any later agent — including the next model generation.
- **Audit-grade traceability.** Every memory write lands in an append-only audit log with an integrity hash, the responsible agent identity, and a timestamp. Compliance and incident reviews finally have a real artifact to inspect.
- **Vendor neutrality preserved.** Memory is held in your PostgreSQL, ranked by deterministic mathematics. Switching model providers does not invalidate the corpus.

Where the cost line bends

Without memory, the marginal cost of every additional agent session is roughly constant — the agent re-pays the discovery tax each time. With memory, the marginal cost trends downward asymptotically: most of the durable architectural facts have already been captured, and the agent retrieves them instead of re-discovering them. The break-even point arrives quickly (empirically within the first dozen sessions on a non-trivial codebase) and the savings compound from then on.

3. System Architecture Overview

Five subsystems, one shared persistence plane.

tapps-brain composes five subsystems behind a thin transport layer: an agent facade, a memory store, a retrieval pipeline, an optional cross-agent Hive, and a quality loop. All durable state — private memory, Hive, Federation, the Knowledge Graph, the audit log, diagnostics history, and feedback events — lives in PostgreSQL. Agents never address Postgres directly; they call the agent facade or the MCP tool surface.

Subsystems

Agent facade	Five-method surface — remember, recall, forget, learn-from-success, learn-from-failure. Agents reason at this level only.
Memory store	Coordinates safety, conflict detection, write policy, persistence, propagation, consolidation, decay, and audit. Synchronous core; optional async wrapper.
Retrieval pipeline	Lexical (BM25 over tsvector candidates) and dense (pgvector HNSW) channels merged with Reciprocal Rank Fusion. Optional FlashRank cross-encoder rerank.
Hive (optional)	Cross-agent memory sharing inside a project. Declarative group membership, expert auto-publishing, LISTEN/NOTIFY change feed.
Quality loop	Deterministic feedback signals, EWMA-based anomaly detection, circuit breaker on retrieval quality, Bayesian confidence updates.

Inbound surfaces

- **MCP Streamable HTTP** on :8080 — the primary agent surface. Tool families *brain_**, *memory_**, *kg_**, *hive_**, *feedback_**.
- **REST API** on :8080 (*/v1/**) — equivalent capability for clients that cannot speak MCP. Parity is mandated by ADR-008.
- **Operator MCP** on :8090 — maintenance surface for SREs (garbage collection, consolidation, migrations, hive restore). Gated by a separate auth token.
- **Python facade** — same engine, in-process. Embed in your agent loop or your tests.
- **CLI** — operator and developer entry point over the same service.

4. C4 — System Context

Who talks to tapps-brain, and across which boundaries.

Coding agents (Claude Code, Cursor, AgentForge, Ralph loop) speak MCP and REST. A single human operator role uses the CLI for maintenance. The brain itself is one container that owns a Postgres cluster, optionally emits OpenTelemetry traces, and optionally calls out to Context7 for library documentation lookups during validation paths. Downstream consumers (AgentForge BrainBridge) mediate their own agents through tapps-brain, never giving raw DB credentials to those agents.

tapps-brain — System Context

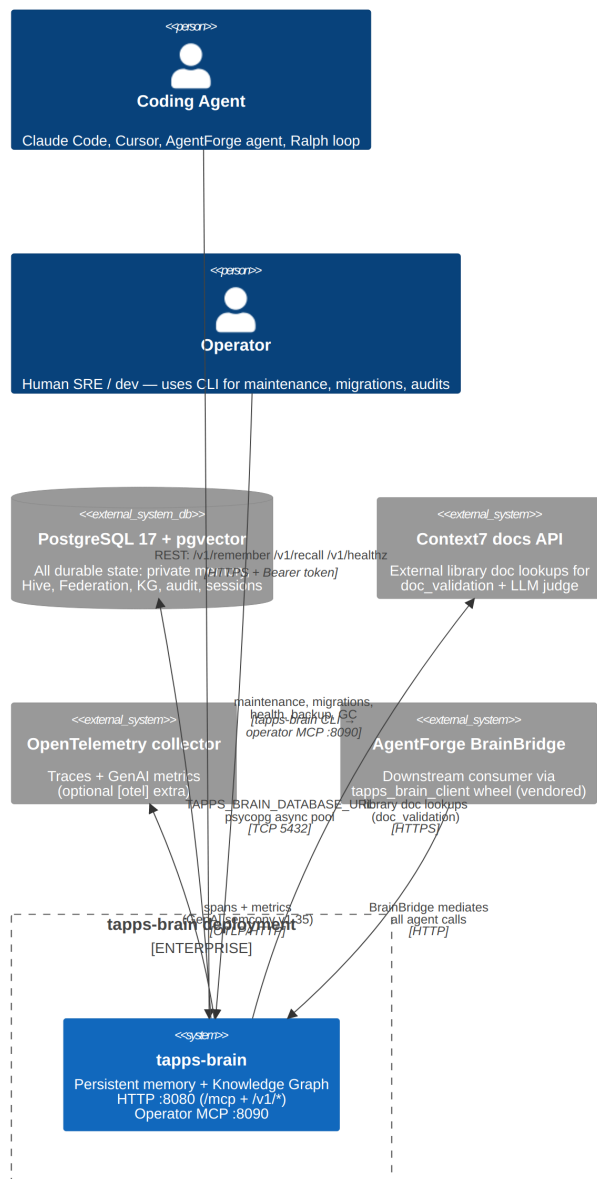


Figure 4.1 — C4 system context. Dashed boundaries are external; the brain runs as a single container with one Postgres dependency.

5. C4 — Container View

How a request lands and reaches the persistence plane.

Inside the container, a FastAPI ASGI adapter authenticates the caller, applies the profile gate, and dispatches either to the REST router or to the FastMCP streamable endpoint. Both transports terminate in the same set of pure service functions, which in turn talk to the memory store, the retriever, and the experience-event recorder. The experience-event recorder is the only writer that holds a multi-table transaction open — every other write goes through the memory store's write-through cache.

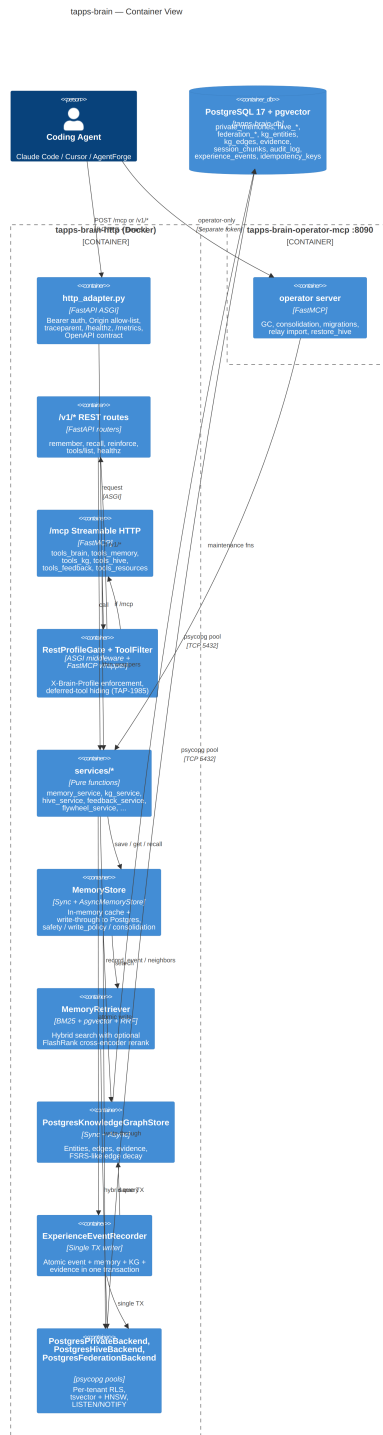


Figure 5.1 — Container view. The profile gate (TAP-1985) decides which tool surface is visible per session.

6. C4 — MCP Server Components

The agent-facing tool layer is a thin wrapper over the services package.

Tools are organised by capability family — brain, memory, knowledge graph, hive, feedback, maintenance, resources, agent registry. Every tool body delegates to a function inside *services/*; the MCP layer adds nothing but envelope deserialisation, profile gating, and JSON serialisation. This separation is what makes the REST and MCP surfaces enforceably equivalent.

MCP Server — Components

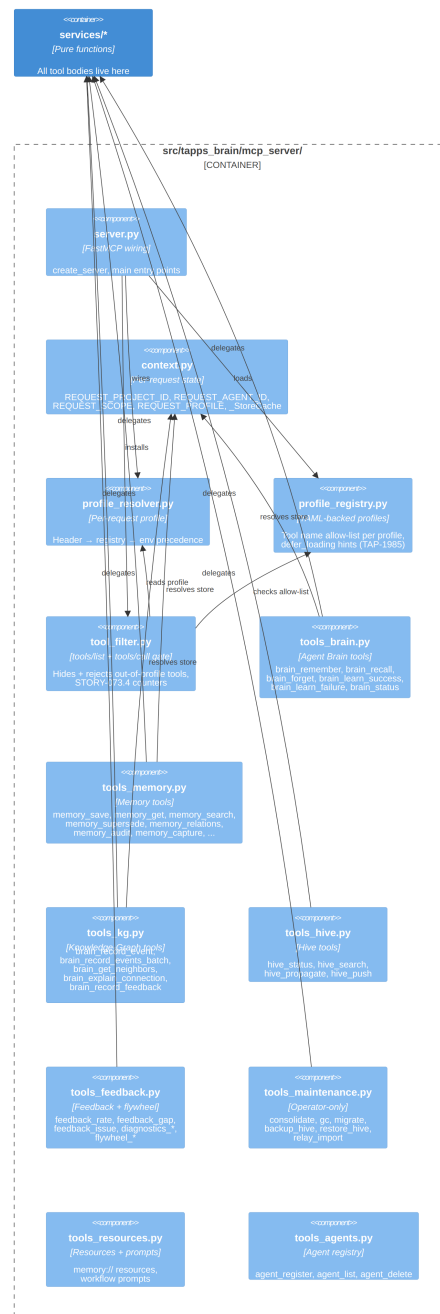


Figure 6.1 — MCP server components. The `tool_filter` enforces the per-profile allow-list at both `tools/list` and `tools/call`.

7. C4 — Services Layer

Pure functions, transport-agnostic, end-to-end testable.

The services package is the architectural seam of the system. Every business operation has exactly one implementation here; transports (REST, MCP, CLI, library) are thin parameter adapters. The contract is: services depend on protocols, never on transports, and the test suite exercises them directly without an HTTP or MCP loop.

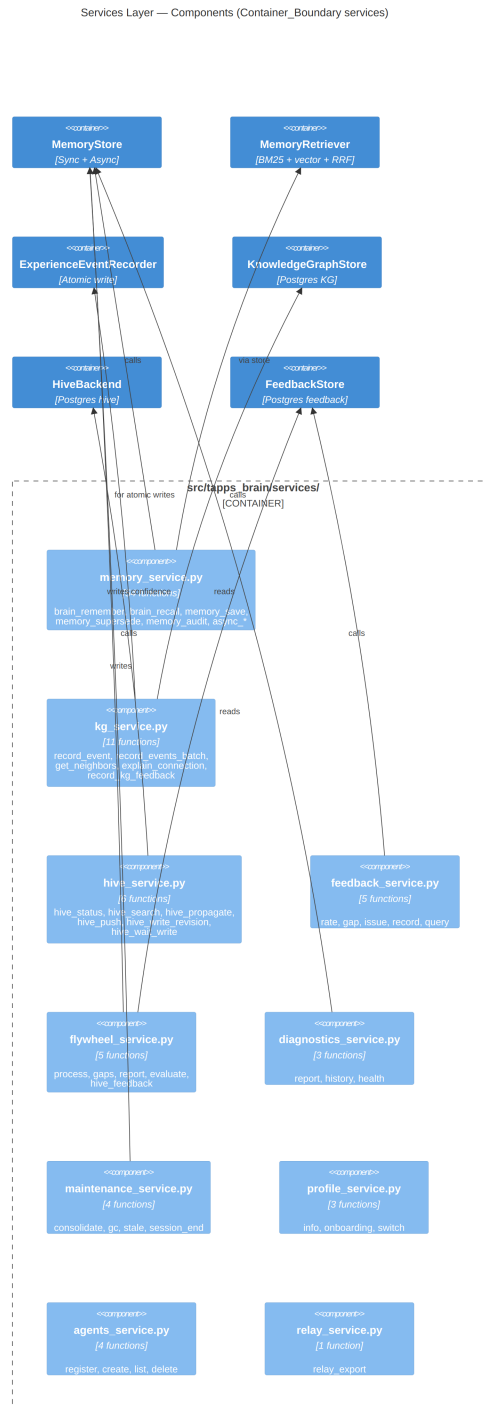


Figure 7.1 — Services layer composition. The memory service dominates the surface area; the experience-event recorder owns the atomic Knowledge Graph write path.

9. Domain Data Model

The MemoryEntry is the spine — every persisted row maps to one.

The Pydantic *MemoryEntry* carries every field an agent or operator ever needs to reason about a memory: the content, its tier, its confidence, its source attribution, its tags, its lifecycle status, its temporal validity window, its FSRS-style stability and difficulty, its integrity hash, and its agent scope. The Knowledge Graph views (*KGEntityView*, *KGEdgeView*, *KGEvidenceView*) project the entity / edge / evidence tables into a recall-friendly shape — the same recall response can carry ranked memories, the entities mentioned, the edges that connect them, and the supporting evidence quotes, all in one document.

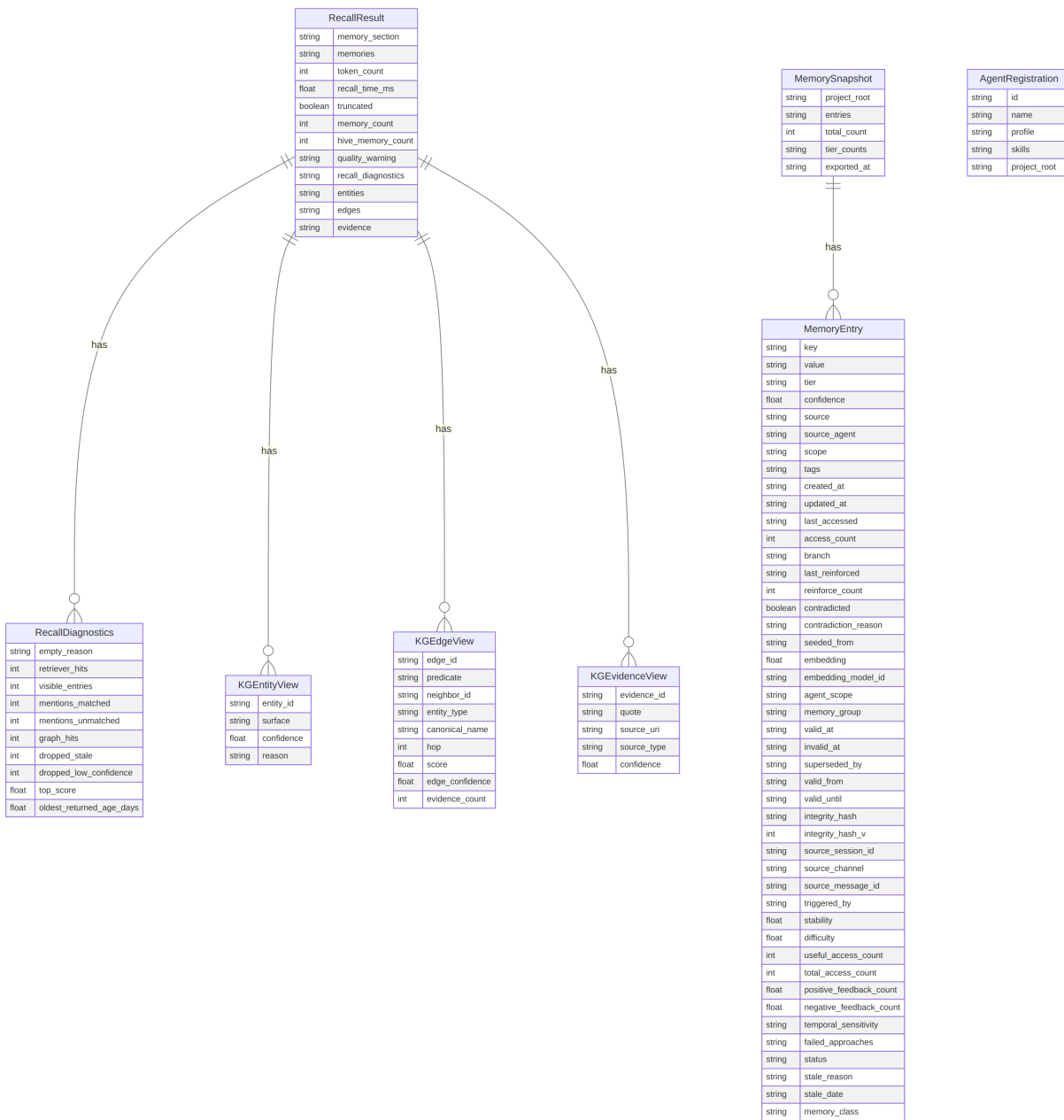


Figure 9.1 — Pydantic entity-relationship overview. *RecallResult* composes *MemoryEntry*, the KG views, and diagnostics into the single document returned to agents.

10. Error Taxonomy

Stable wire codes over a semantic SDK hierarchy.

Errors are first-class. The wire taxonomy maps each fault to a stable code, an HTTP status, a JSON-RPC code, and a retry policy. Clients catch by intent — auth errors stop and re-token, transient errors are retried by the SDK and escalated otherwise, transport errors degrade gracefully. The categorical taxonomy means a new failure mode can be introduced without breaking existing clients: they continue to match on the parent exception class.

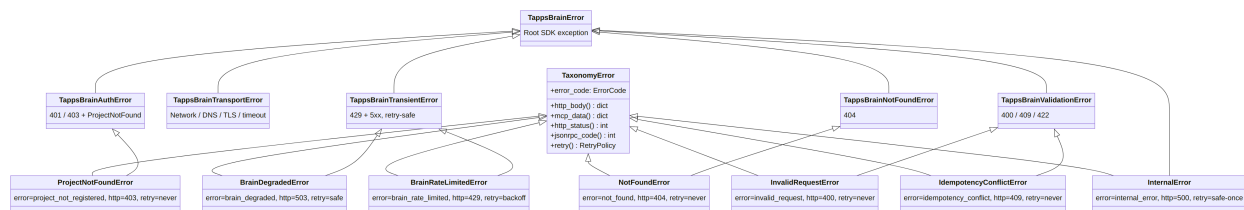


Figure 10.1 — Stable wire taxonomy (top half) layered over the semantic SDK hierarchy (bottom half). Catch by intent, not by string match.

11. Flow — Recall

How an agent question turns into ranked memories.

Recall is the high-frequency operation. An agent sends a natural-language query; the system extracts entity mentions, runs the lexical and dense channels in parallel, fuses their rankings, optionally reranks the top candidates with a local cross-encoder, applies lazy decay and safety filters, packs the result inside a token budget, and returns ranked memories plus diagnostics. Nothing in this path calls an external LLM. Everything is reproducible from the inputs.

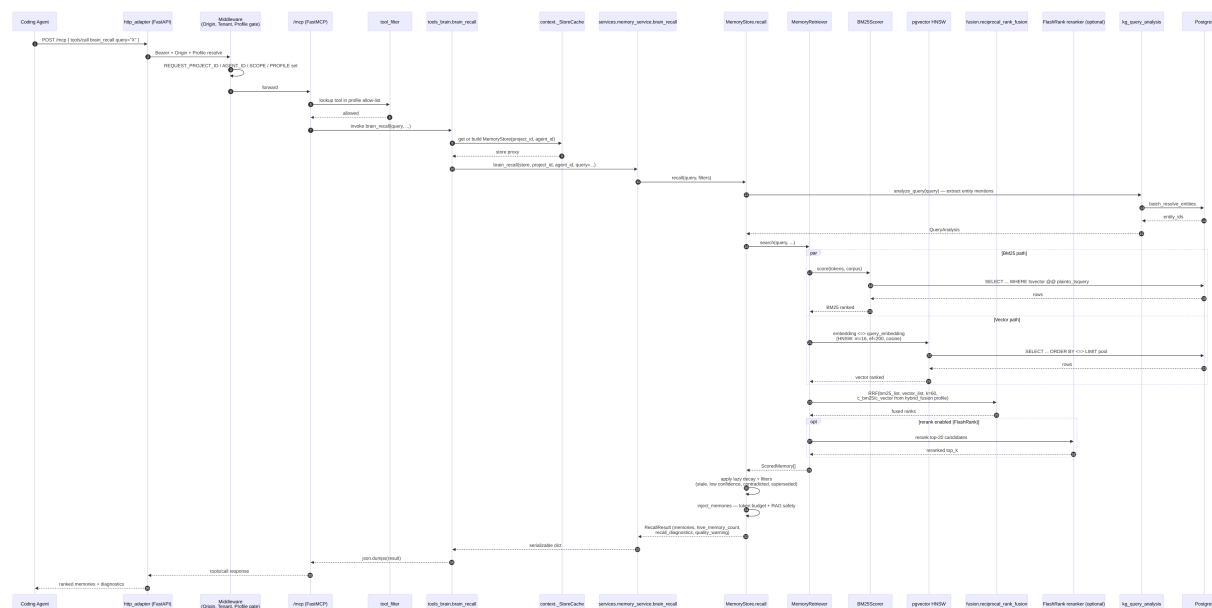


Figure 11.1 — End-to-end recall. Parallel BM25 and pgvector channels merged by Reciprocal Rank Fusion, with optional FlashRank rerank.

13. Flow — Atomic Knowledge Graph Event

One transaction, five tables, no partial writes possible.

When an agent records an experience event — a debug session, a refactor, a successful deploy — the experience-event recorder writes the event row, the optional memory row, the entity upserts, the edge upserts, and the evidence rows in a single psycopg transaction. Edges without supporting evidence have their confidence capped at 0.4 (ADR-012); the system *never* ranks an unsupported claim ahead of a supported one. Any failure rolls back the entire transaction, including the event itself, so a partial graph is structurally impossible.

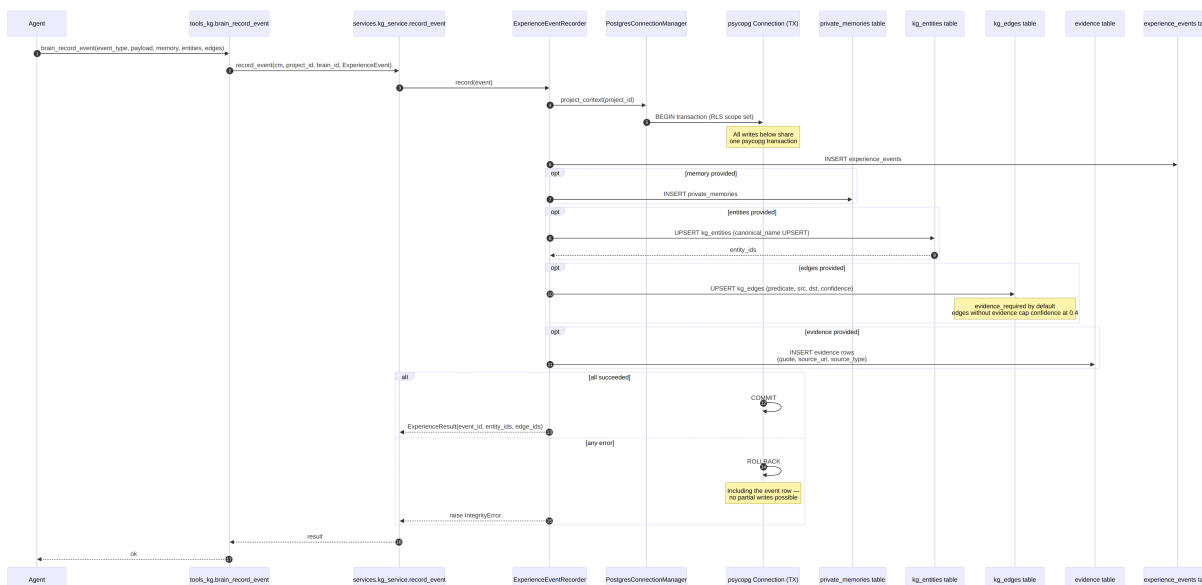


Figure 13.1 — Atomic experience-event write. All-or-nothing across five tables under one Postgres transaction; RLS is set on the transaction scope.

14. Flow — Hive Propagation

From a domain-scoped save to a subscriber's recall, with namespace isolation.

Hive is opt-in cross-agent sharing inside a project. When agent A saves with `agent_scope = "domain"` and a memory group, the propagation engine writes the row to the Hive namespace and fires a Postgres LISTEN/NOTIFY pulse on the change channel. Other agents that have subscribed to that channel get the wake-up; agents that simply recall later pick the row up through the normal namespace-aware search. Local results are merged with Hive results at recall time, weighted (default 0.8 for Hive); the agent never has to think about where the memory came from.

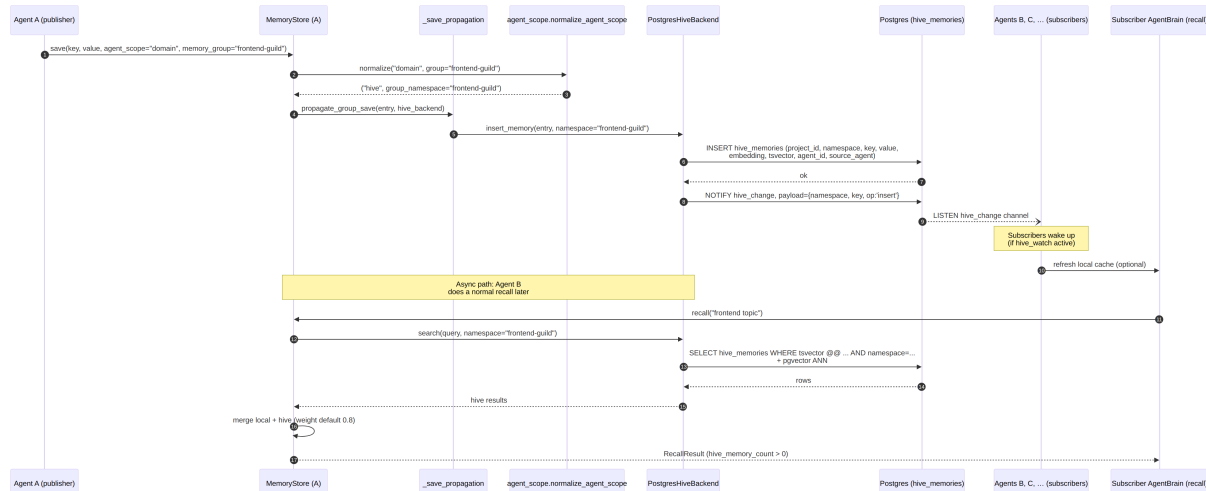


Figure 14.1 — Hive propagation. The publish side is a single Postgres insert plus a NOTIFY; subscribers wake or simply find the row on their next recall.

15. Mathematical Foundations

The deterministic formulas behind every ranking, every decay, every merge.

15.1 Okapi BM25

Lexical relevance is scored with Okapi BM25, the de-facto standard for keyword retrieval. For a query Q tokenised into terms $q_1, \dots, q_n$ and a candidate document D with length $|D|$, BM25 sums per-term contributions:

$$\text{BM25}(D, Q) = \sum_i \text{IDF}(q_i) \cdot [f(q_i, D) \cdot (k_1 + 1)] / [f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D| / \text{avgdl})]$$

where $f(q_i, D)$ is the term frequency, avgdl is the average document length over the corpus, and $k_1 \sim 1.2$, $b \sim 0.75$ are the standard saturation and length-normalisation parameters. IDF is the inverse-document-frequency smoothed against the full project corpus, not just the candidate slice — this is the difference between BM25 done well and BM25 done lazily, and it is why tapps-brain builds the in-process IDF index over the entire store.

15.2 Reciprocal Rank Fusion (RRF)

Lexical and dense channels disagree often — and that disagreement is signal. RRF fuses two (or more) ranked lists by summing the reciprocal of each candidate's rank in each list, with a small constant k that dampens early-rank dominance:

$$\text{RRF}(d) = \sum_{r \in R} w_r / (k + \text{rank}_r(d))$$

where R ranges over the channels (lexical, dense, optionally graph), $\text{rank}_r(d)$ is the document's 1-indexed rank inside channel r , and w_r is a per-query weight chosen deterministically by short, explainable heuristics (query length, query type, mention density). The constant k defaults to 60 — large enough to keep mid-rank candidates competitive, small enough to honour true top-1 agreement. RRF is rank-only and scale-free: it does not care whether BM25 returned an *ts_rank_cd* of 0.42 or a vector cosine of 0.18 — it cares which document came first.

15.3 Composite scoring

After fusion, the system applies a small composite score that turns relevance into a ranking that also respects the human signal — how confident an entry is, how recently it has been touched, and how often it has been useful in the past:

$$\text{score}(d) = 0.40 \cdot \text{relevance}(d) + 0.30 \cdot \text{confidence}(d) + 0.15 \cdot \text{recency}(d) + 0.15 \cdot \text{frequency}(d)$$

Each term is normalised to $[0, 1]$. *Relevance* is the fused retrieval score; *confidence* is the per-entry value updated by feedback events; *recency* is the decayed time-since-last-touch; *frequency* is the access-count proxy. Per-source trust multipliers (human edits trusted over agent-extracted facts) and an optional graph-centrality contribution are applied after composition. The weights are profile-tunable, but the formula is fixed — there is no LLM in the loop, ever.

15.4 Exponential decay (lazy)

Memories lose ranking weight as they age, but only on read — there are no wall-clock TTL jobs (ADR-002). For an entry at age t with tier half-life h , the decayed effective recency / confidence factor is:

$$\phi(t) = \exp(-\ln(2) \cdot t / h)$$

Tier half-lives match the natural staleness of the content: architectural decisions (180 days) survive long enough to influence multiple quarterly cycles; coding patterns (60 days) age out across team rotations; procedural facts (30 days) drift with toolchain updates; conversational context (14 days) is intentionally short-lived. Per-entry *temporal_sensitivity* (high / medium / low) multiplies the tier half-life when the content genuinely demands a different cadence.

15.5 FSRS-style stability and difficulty

Beyond tier decay, every entry carries two FSRS-inspired fields: *stability* (how long the entry is expected to remain useful) and *difficulty* (how easily it is forgotten under access pressure). On a useful access, stability grows by a difficulty-dampened multiplier; on a contradicted access, it collapses toward the minimum. This is the same insight that powers spaced-repetition systems, applied to memory entries:

$$S' = S \cdot (1 + \alpha \cdot (1 - D)) \quad \text{and} \quad D' = D + \beta \cdot (1 - \text{feedback})$$

Stability and difficulty refine the recency term — entries with high stability decay more slowly than their tier alone would predict, and high-difficulty entries decay faster. The net effect: the recall pool concentrates around the entries that have actually proved useful across many sessions, without operator intervention.

15.6 Bayesian-style confidence update

User and agent feedback events update per-entry confidence using a small Bayesian-style smoother. With n_+ positive and n_- negative events and a smoothing prior m , the posterior estimate is:

$$c = (n_+ + m \cdot c_0) / (n_+ + n_- + m)$$

where c_0 is the prior (typically the source-trust default for the ingest channel) and m shrinks early evidence toward the prior. A handful of negative ratings can demote an entry below the injection floor; a sustained pattern of useful accesses raises confidence above the consolidation threshold. The flywheel service maintains this curve in the background; it never blocks a recall.

15.7 Bloom filter false-positive bound

The save path uses a Bloom filter to skip full conflict detection on definitely-not-seen writes. For a filter of m bits, n expected items, and k hash functions, the false-positive probability after insertion is:

$$p \approx (1 - e^{-k \cdot n / m})^k$$

Default sizing targets $p \sim 1\%$ at the configured *expected_items* load. A false positive simply re-runs the existing full conflict check; a false negative is impossible, so the filter never causes a missed duplicate. The win is on the common case: a fresh fact bypasses similarity scoring entirely.

15.8 Jaccard / TF-IDF for deterministic consolidation

Auto-consolidation merges sufficiently similar entries without any LLM call. Pairwise similarity is the maximum of a token-set Jaccard score and a TF-IDF cosine, with an optional vector-cosine tie-breaker when embeddings are present:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \max(J(\mathbf{a}, \mathbf{b}), \cos_{\text{tfidf}}(\mathbf{a}, \mathbf{b})) \text{ with } J(\mathbf{a}, \mathbf{b}) = |\mathbf{T}_{\mathbf{a}} \cap \mathbf{T}_{\mathbf{b}}| / |\mathbf{T}_{\mathbf{a}} \cup \mathbf{T}_{\mathbf{b}}|$$

When the maximum crosses the profile threshold, the merge proceeds: the union of tags, the higher confidence, the earlier *valid_from*, the freshest *last_accessed*. Every merge is logged in the audit table with the source row IDs, so any merge can be undone deterministically.

15.9 pgvector HNSW geometry

Dense recall runs over a Hierarchical Navigable Small Worlds (HNSW) graph in pgvector. The index uses $m = 16$ (out-degree per node), *ef_construction* = 200 (build-time candidate breadth), and cosine distance. Search is approximate but bounded: expected recall above 95% against the exact KNN baseline at a fraction of the latency, with no rebuild step after bulk loads. The upgrade from the legacy IVFFlat index (migration 002) delivered roughly 1.5× faster ANN at comparable quality.

16. Data Stores and Schema

One Postgres, three logical planes, six migrations on private memory.

Every durable store lives in PostgreSQL (ADR-007). Per-agent isolation is enforced by a (*project_id*, *agent_id*) composite key on every row, not by separate database files. The three logical planes — private memory, Hive, Federation — can share one Postgres or be split across three by setting the optional *TAPPS_BRAIN_HIVE_DSN* and *TAPPS_BRAIN_FEDERATION_DSN* variables.

Logical planes

Private memory	Per-agent isolated store. Table <i>private_memories</i> , keyed by (<i>project_id</i> , <i>agent_id</i> , <i>key</i>). Default capacity 5,000 entries per project, profile-configurable.
Hive	Cross-agent shared memory inside a project. Tables <i>hive_memories</i> , <i>hive_groups</i> , <i>hive_group_members</i> , <i>hive_write_notify</i> . LISTEN/NOTIFY change feed.
Federation	Cross-project memory sharing. Tables <i>federated_memories</i> and <i>federation_meta</i> . Publish / subscribe / pull only; no automatic background replication.
Knowledge Graph	Entities, edges, evidence, aliases, experience events. Edges without evidence have confidence capped at 0.4 (ADR-012).
Audit log	Append-only Postgres table; every save, delete, archive, reinforce, consolidation, and undo lands here with a timestamp and the agent identity.

Search index strategy

Lexical recall uses a *tsvector* column with weighted *setweight* (**A** on the key, **B** on the value, **C** on the tags) and a GIN index. Dense recall uses a 384-dimensional *vector* column on the same row with an HNSW index. The two indexes are kept in lock-step on write; on read, both are queried in parallel and fused by RRF. The pure-Python BM25 layer in *bm25.py* re-ranks the lexical candidates using full-corpus IDF, so ranking fidelity does not depend on *ts_rank_cd* alone.

Eviction policy

When a new key arrives and the project is already at *limits.max_entries*, the memory store evicts exactly one row — the entry with the lowest *stored* confidence (not the decay-adjusted effective confidence). Ties resolve by dict iteration order. If *limits.max_entries_per_group* is set, per-bucket caps apply first, with the bucket-fair global eviction preferring the incoming row's bucket on the secondary cap breach. Evicted rows are *logged*, not lost: they pass through *gc_archive* before final removal, and the audit log records the eviction reason.

17. Security & Threat Model

STRIDE applied to a memory system that holds the team's accumulated knowledge.

tapps-brain is a high-value target: it stores the architectural rationale, debugging history, and coding patterns that an organisation's agents rely on to act correctly. Compromise the brain and you compromise every downstream agent decision. The threat model is built around STRIDE — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — with each row tied to an existing control in the codebase.

17.1 Per-category mitigations

Spoofing	Agent identity is session-bound at the store layer; the <i>tapps_runtime</i> Postgres role has DML only, scoped to the caller's <i>project_id</i> via RLS. DSNs are never logged in plaintext.
Tampering	Prompt-injection patterns detected and blocked on ingest and before injection (<i>safety.py</i>). Cross-agent writes constrained by the propagation engine's scope rules; schema changes require the privileged <i>tapps_migrator</i> role.
Repudiation	Every mutation appends to an immutable <i>audit_log</i> table with timestamp, agent identity, and operation type. Per-entry <i>integrity_hash</i> (HMAC v2) detects after-the-fact alteration.
Information disclosure	All shared tables filtered by <i>project_id</i> at the app layer and optionally enforced again by Row-Level Security. Telemetry forbids raw memory bodies, query strings, and PII; logs strip or hash sensitive fields.
Denial of service	Per-project entry caps (default 5,000), bounded psycopg connection pool, rate-limited write path, deterministic and bounded auto-consolidation. <i>/ready</i> exposes pool saturation.
Elevation of privilege	Runtime and migrator roles separated. Operator MCP tools (GC, consolidation-undo, diagnostics reset) gated behind an explicit flag and a separate auth token. ADR-008 mandates MCP and library parity before any new HTTP route is added.

Encryption at rest

At-rest encryption is the storage layer's responsibility — Percona *pg_tde* ($\geq 2.1.2$) or cloud-managed TDE. Application code does not handle keys, never sees the key material, and never logs the DSN with credentials. Key rotation is documented as an operator runbook; the application is unaffected.

18. Observability & the Quality Loop

Knowing when the recall pool is healthy, and reacting when it isn't.

Quality is not asserted; it is measured. Three subsystems cooperate. **Feedback** captures typed events (useful / not-useful / contradicted / stale) when agents react to a recall. **Diagnostics** turns feedback into a composite scorecard, runs an EWMA (exponentially-weighted moving average) anomaly detector over the recent recall stream, and trips a circuit breaker when the recall quality degrades. **Flywheel** translates feedback into per-entry confidence updates over time, generates a markdown report on demand, and optionally consults an LLM judge in offline mode for deeper quality scoring.

18.1 EWMA anomaly detector

For a quality signal x_t at step t (for example, the rolling fraction of recalls that received a useful-rating), the EWMA estimate is:

$$\mu_t = \lambda \cdot x_t + (1 - \lambda) \cdot \mu_{t-1}$$

with smoothing λ typically in 0.1–0.3. An anomaly fires when μ_t crosses a guard band derived from a baseline window. The circuit breaker then attaches a *quality_warning* to subsequent *RecallResult* objects, so agents (and operators) know the recall stream is degraded without an out-of-band notification.

18.2 Telemetry surface

OpenTelemetry traces and metrics are emitted on the hot paths. Span names are constants (no string literals are accepted in code review): *tapps_brain.remember*, *tapps_brain.recall*, *tapps_brain.search*, *tapps_brain.hive.propagate*, *tapps_brain.hive.search*. Span and metric attributes are restricted to a bounded enum allow-list; raw memory bodies, query strings, session IDs, and agent IDs are forbidden. The HTTP adapter extracts W3C *traceparent* headers so spans are correctly parented to upstream callers.

19. Concurrency and Scale

How 200+ concurrent agents share one Postgres without contention.

The scale model rests on two ideas. First, every private-memory row carries a *(project_id, agent_id)* key, so agents share the database but not the rows. There is no cross-agent lock contention for private memory — two hundred agents read and write two hundred disjoint row sets. Second, the shared planes (Hive, Federation, Knowledge Graph) rely on PostgreSQL's MVCC for concurrency. Connection pooling (*psycopg_pool*) bounds the in-flight transaction count; the pool's min and max are environment-tunable, and the */ready* probe exposes saturation so an orchestrator can back off before the pool is exhausted.

Locking inside one agent

Inside a single agent, the memory store uses a plain (non-reentrant) *threading.Lock*. One mutating or read-heavy store operation runs at a time per agent per process. The lock is acquired before any persistence call, never inside it, which eliminates the classic re-entry deadlock. An optional lock timeout protects agents from a pathological writer.

No async core

The core is synchronous by design. Async surfaces exist (*AsyncMemoryStore*, the async *psycopg* backends, the async Knowledge Graph store) but they are thin *asyncio.to_thread* wrappers around the sync core. This keeps the determinism guarantees and the test surface manageable; async is offered to callers who need it, not imposed on callers who don't.

20. Deployment Topology

One compose file, two long-running containers, one one-shot migrator.

tapps-brain ships as a unified Docker stack. Three containers compose the cluster: the database (*pgvector/pg17*), the brain itself (HTTP + MCP on :8080, operator MCP on :8090), and a one-shot migrator sidecar that runs schema migrations and creates the DML-only runtime role on first boot. An optional *tapps-visual* dashboard polls the brain's */snapshot* endpoint and renders a live operator view.

tapps-brain-db	PostgreSQL 17 with the <i>vector</i> extension pre-loaded. Single instance; operator-managed backup cadence. <i>pg_tde</i> at the storage layer if required.
tapps-brain-migrate	Sidecar image; entrypoint applies private, Hive, and Federation migrations, creates the <i>tapps_runtime</i> DML-only role, then exits. Re-run on upgrade.
tapps-brain-http	Long-running brain process. Serves MCP Streamable HTTP on <i>/mcp</i> , REST on <i>/v1/*</i> , health on <i>/healthz</i> , metrics on <i>/metrics</i> , operator MCP on :8090. Bearer auth on every write route.
tapps-visual (optional)	nginx-served dashboard. Polls <i>/snapshot</i> every 30 s with a LIVE / STALE / OFFLINE / ERROR badge. No file-load fallback.
Secrets	All credentials live in <i>docker/.env</i> (DB password, runtime password, auth and admin tokens). The template is committed; the secrets file is not.

21. Closing — Business Value Summary

The shape of the argument, restated.

An AI engineering organisation can spend its budget on context every session, or it can spend it once and amortise. tapps-brain is the second option. By converting the ambient knowledge that an agent generates — debug findings, architectural rationale, coding conventions, workflow patterns — into a ranked, decayed, deterministically retrievable corpus, the system turns repeat work into accumulated capital.

The technical commitments that make this work are conservative on purpose. PostgreSQL is the persistence plane (one well-understood operational surface, row-level multi-tenancy, MVCC concurrency, mature backup tooling, no proprietary vector lock-in). Determinism is the contract (every recall is reproducible, every merge is logged and undoable, every formula is published). MCP and REST are the transports (the two surfaces that today's IDE-side agents already speak, plus an ADR-enforced parity rule so adding one tool to one surface never silently leaves the other behind).

What you get on day one

An agent that remembers what it learned yesterday. A team that sees its conventions captured the moment they emerge. An audit trail with an integrity hash on every write. A retrieval pipeline whose ranking can be reproduced from the inputs and explained to an auditor. And a deployment topology a single SRE can operate without paging.

What you get over time

A monotonically improving corpus. Confidence updates from real feedback bring the useful entries to the top of every recall, while exponential decay and deterministic consolidation quietly remove the rest. Cross-agent sharing (Hive) lets specialists publish to a group, and cross-project sharing (Federation) lets a multi-project organisation amortise the corpus across its entire portfolio.

Persistent memory is not a feature on an agent; it is a strategic asset of the team that operates the agent. tapps-brain is the runtime that turns the asset on.

A. Appendix — Module Dependency Overview

Auto-generated package graph (50 of 138 modules shown).

The dependency graph below is generated from the live source tree by the *docs-mcp* diagram tool and is included here for completeness. It is not intended for end-user reading; it serves as an architectural integrity check — presentation modules (pink) depend on services (teal), services depend on the core (teal), core depends only on ports and protocols (grey), and concrete data adapters (purple) sit at the bottom of the graph.

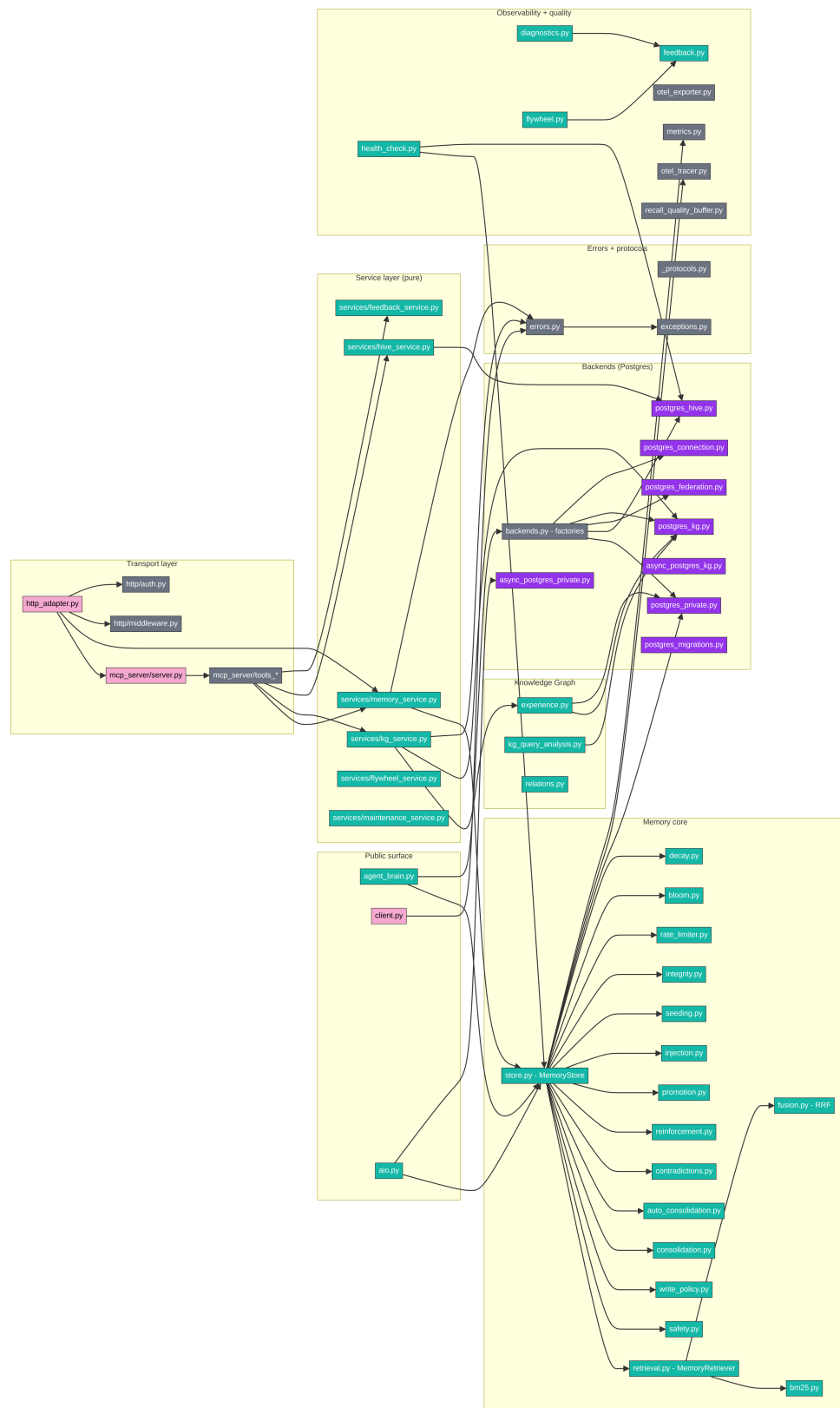


Figure A.1 — Module dependency overview (truncated). Colour coding: pink = presentation, teal = business / domain, purple = data, grey = infrastructure.